

# 末尾呼び出し最適化 — 再帰をループに変える

## 今日のゴール

`return f(...);` の形で自分自身を呼ぶ再帰を、**ジャンプ**に変換する。100 万回の再帰でもスタックがあふれないコンパイラにする。

## 1 この回の位置づけ

- バックエンド発展シリーズの第 3 回（選択制）。前提はコマ 8（関数呼び出し）まで
- コマ 8 の資料で「扱わない」とした**末尾呼び出し最適化**の回収
- `mycc.py` は書き換ええない。ラッパー `tccc.py`（完成品）が差し込む
- 編集するのは `tailcall.py` の 2 つの関数だけ。B1・B2 とは独立している

## 2 末尾呼び出しとは

関数の**最後の仕事**が別の関数の呼び出しであるとき、それを末尾呼び出しという。

```
int sum_to(int n, int acc) {  
    if (n == 0) { return acc; }  
    return sum_to(n - 1, acc + n);    /* 末尾呼び出し */  
}
```

対して、次は末尾呼び出しではない。

```
int fact(int n) {  
    if (n == 0) { return 1; }  
    return n * fact(n - 1);    /* 呼び出しの後に掛け算が残っている */  
}
```

違いは「呼び出しから戻ってきた後に、まだやることがあるか」である。やること残っていないなら、呼び出し元のフレームはもう用済みである。

### 3 なぜジャンプに変えられるのか

`sum_to` が `sum_to` を呼ぶとき、いまのコンパイラは新しいフレームを積む。

```
sum_to(1000000, 0) のフレーム
  sum_to(999999, 1000000) のフレーム
    sum_to(999998, 1999999) のフレーム
      ... 100 万段 → スタックがあふれる
```

しかし各フレームは、呼び出しから戻った瞬間に「戻り値をそのまま返す」だけである。つまりフレームを積む意味がない。引数だけ新しい値に書き換えて、関数の先頭に戻れば、同じ計算ができる。

```
# return sum_to(n - 1, acc + n);
# 新しい引数を計算して、引数スロットに書き戻し...
sd a0, -24(s0)      # n ← n - 1
sd a1, -32(s0)      # acc ← acc + n
j .Ltc_sum_to       # フレームを積まずに先頭へ戻る
```

`call` が `j` になり、フレームが 1 つも増えない。再帰がループになった。

#### 補足

関数型言語では必須の最適化。Scheme や OCaml では、ループを書く標準的な方法が末尾再帰である。そのため言語仕様のレベルで「末尾呼び出しは必ずジャンプに変換すること」(proper tail call) が要求される。C の規格には要求がないが、gcc や clang は `-O2` で同じ最適化を行う。

## 4 実装の 3 つの注意点

### 4.1 1. どこへ戻るか — プロローグの後

戻り先は関数の先頭ラベルではない。プロローグ（フレーム確保）と引数の保存を飛ばした、本体の直前である。

```
sum_to:
  addi sp, sp, -32      ↗ プロローグ
  sd ra, 24(sp)         |   ここに戻ると
  ...                  |   フレームが増えてしまう
  sd a0, -24(s0)        ↘   引数の保存
.Ltc_sum_to:           ←   ここへ戻る
# 本体
```

ラッパー `tccc.py` が、この位置にラベルを置いて `cg.tc_label` に入れてくれる。

## 4.2 2. 引数は「全部評価してから」書き戻す

```
return sum_to(n - 1, acc + n);
```

第 1 引数を先に `n` のスロットへ書き込んでしまうと、第 2 引数の `acc + n` が新しい `n` を読んでしまう。コマ 3 以来の退避パターンと同じ理由で、すべて評価して退避 → まとめて書き戻すの順にする。

## 4.3 3. 変換してよい形を厳しく判定する

| 形                                      | 変換  | 理由           |
|----------------------------------------|-----|--------------|
| <code>return f(a, b);</code> (f は自分自身) | する  | 純粋な末尾呼び出し    |
| <code>return f(a) + 1;</code>          | しない | 呼び出しの後に演算が残る |
| <code>return g(a);</code> (別の関数)       | しない | 引数スロットの配置が違う |
| 引数が 9 個以上                              | しない | レジスタに収まらない   |

「怪しいものは変換しない」 — B1 と同じく、最適化の基本姿勢である。判定が広すぎると `fixed15` が壊れるので、そこで気づける。

## 5 実装

| Step | 実装対象                           | 内容                |
|------|--------------------------------|-------------------|
| 1    | <code>is_self_tail_call</code> | 変換してよい形かの判定       |
| 2    | <code>gen_tail_call</code>     | 引数の詰め替え + ジャンプの生成 |

```
python3 tccc.py tests/deep_sum.c # 生成結果を見る
python3 tccc.py tests/deep_sum.c | grep -c call # call が減っているか
```

## 6 テスト

```
python3 check.py # Step ごとの単体テスト（未実装は SKIP）
python3 golden.py # 深い末尾再帰 + fixed15
```

`golden.py` は 3 つを続けて確認する。

1. 最適化なしで `tests/deep_sum.c` (100 万回の末尾再帰) が落ちること
2. 最適化ありで同じテストが通ること
3. `fixed15` が壊れていないこと

「落ちていたものが通るようになる」ので、効果が命令数より分かりやすい。

## 7 発展課題

1. 相互末尾再帰: `is_even / is_odd` のような相互再帰は、自分自身ではないので今回の判定では変換されない。別の関数への末尾呼び出しを変換するには何が必要か考える（フレームの解放、引数の渡し方、`ra` の扱い）
2. 末尾位置の拡張: `if (c) return f(x); else return f(y);` の両方が末尾呼び出しであることは、いまの判定でも扱えている。では `return c ? f(x) : g(y);` は？（三項演算子は Core 外だが設計として考える）

3. **fact** を末尾再帰に書き換える: `fact(n)` を累積引数付きの `fact_acc(n, acc)` に書き換え、深い階乗が動くことを確かめる
4. フレームサイズの計測: 最適化あり／なしで `sp` がどこまで下がるかを `gdb-multiarch` で観察する

#### 補足

**コラム: 再帰とループは同じもの。**「末尾再帰はループに変換できる」という事実は、逆に「ループは末尾再帰で書ける」ことも意味する。実際、コマ 6 で作った `while` の生成コードと、この回で作った末尾呼び出しの生成コードはどちらも「変数を更新して先頭へ `j`」という同じ形をしている。言語の見た目が違っても、機械語のレベルでは同じものになる。